

Crypto Concepts

Symmetric encryption,
Public key encryption,
and TLS

Cryptography

Is:

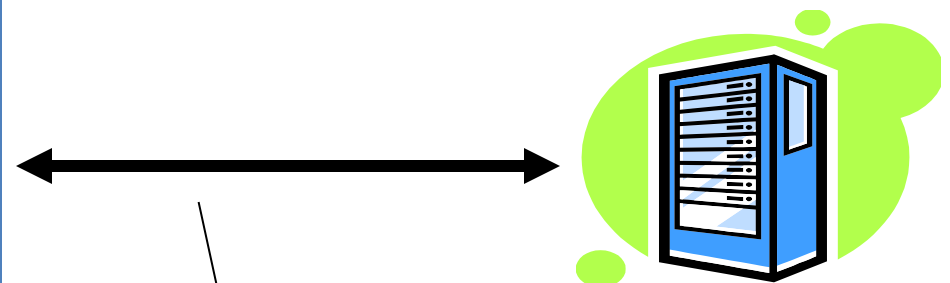
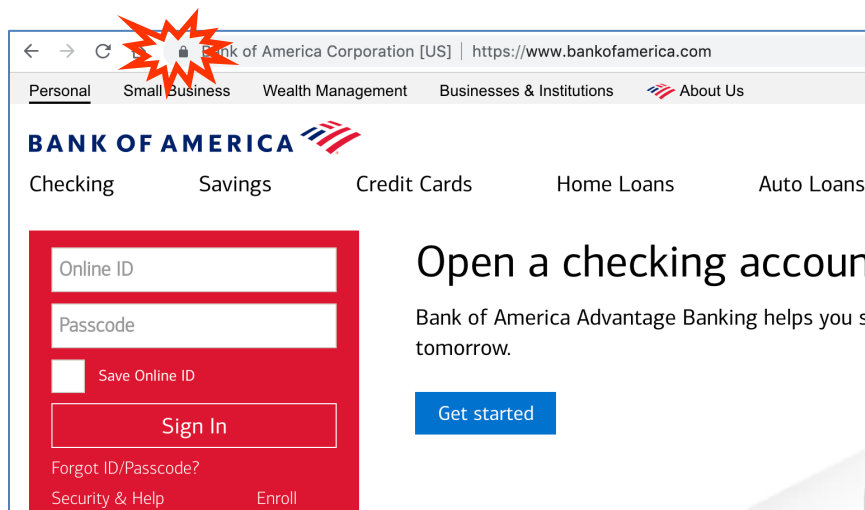
- A tremendous tool for protecting information
- The basis for many security mechanisms

Is not:

- The solution to all security problems
- Reliable unless implemented and used properly
- Something you should try to invent yourself

Goal 1: Secure communication

(protecting data in motion)



no eavesdropping
no tampering

Transport Layer Security / TLS

Standard for Internet security

- Goal: “... provide privacy and reliability between two communicating applications”

Two main parts

1. Handshake Protocol: **Establish shared secret key using public-key cryptography**
2. Record Layer: **Transmit data using negotiated key**

Our starting point: Using a key for encryption and integrity

Diffie-Hellman key exchange

- Algorithm

Global Public Elements

q	prime number
α	$\alpha < q$ and α a primitive root of q

User A Key Generation

Select private X_A	$X_A < q$
Calculate public Y_A	$Y_A = \alpha^{X_A} \bmod q$

User B Key Generation

Select private X_B	$X_B < q$
Calculate public Y_B	$Y_B = \alpha^{X_B} \bmod q$

Calculation of Secret Key by User A

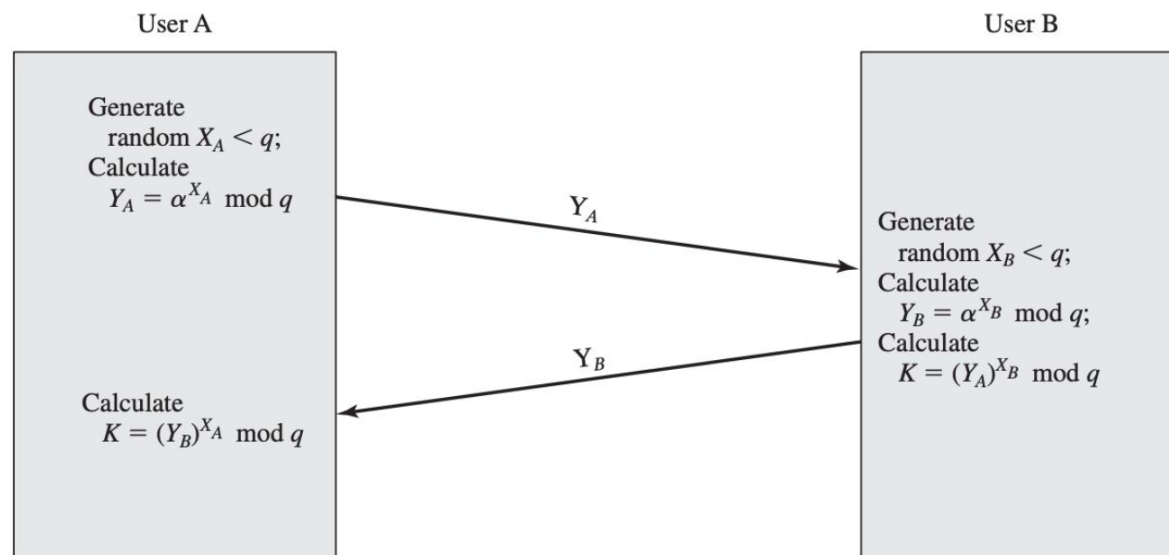
$$K = (Y_B)^{X_A} \bmod q$$

Calculation of Secret Key by User B

$$K = (Y_A)^{X_B} \bmod q$$

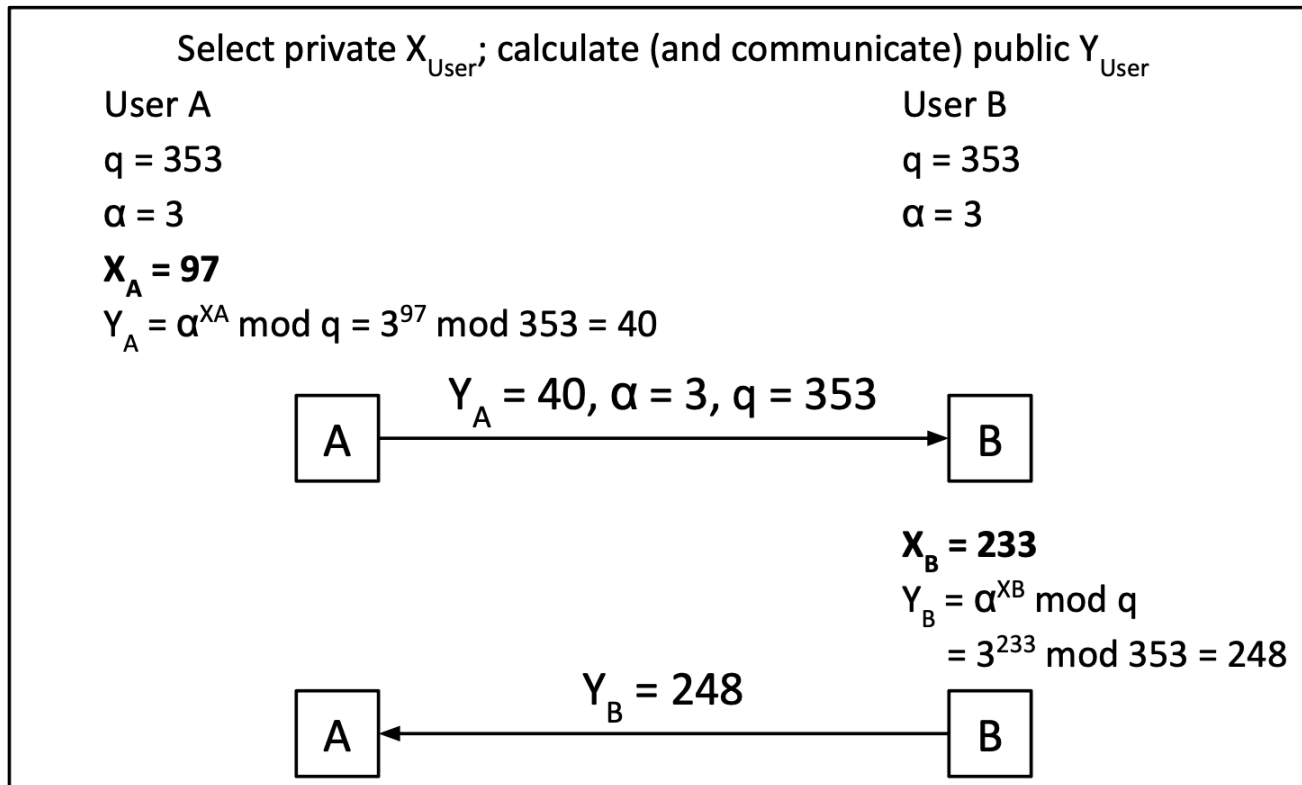
Diffie-Hellman key exchange

- Key exchange



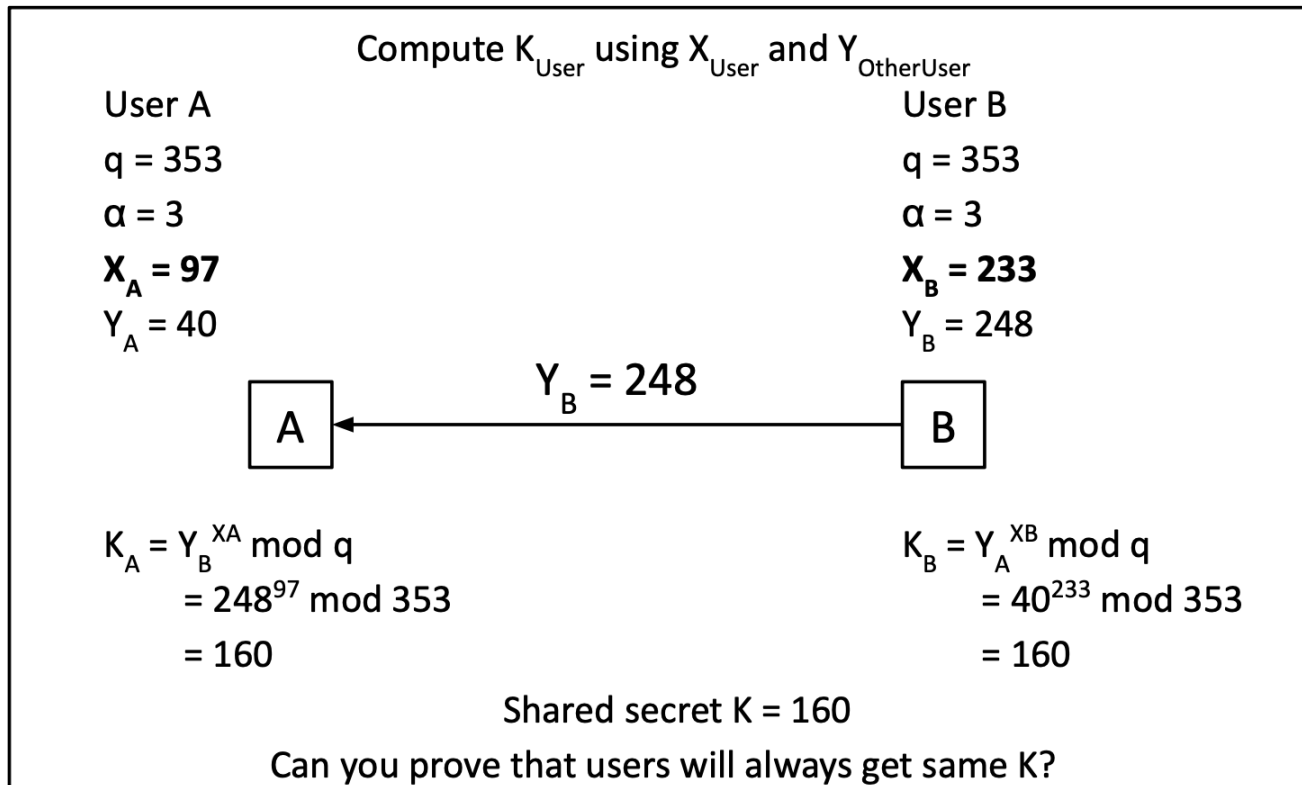
Diffie-Hellman key exchange

- Example



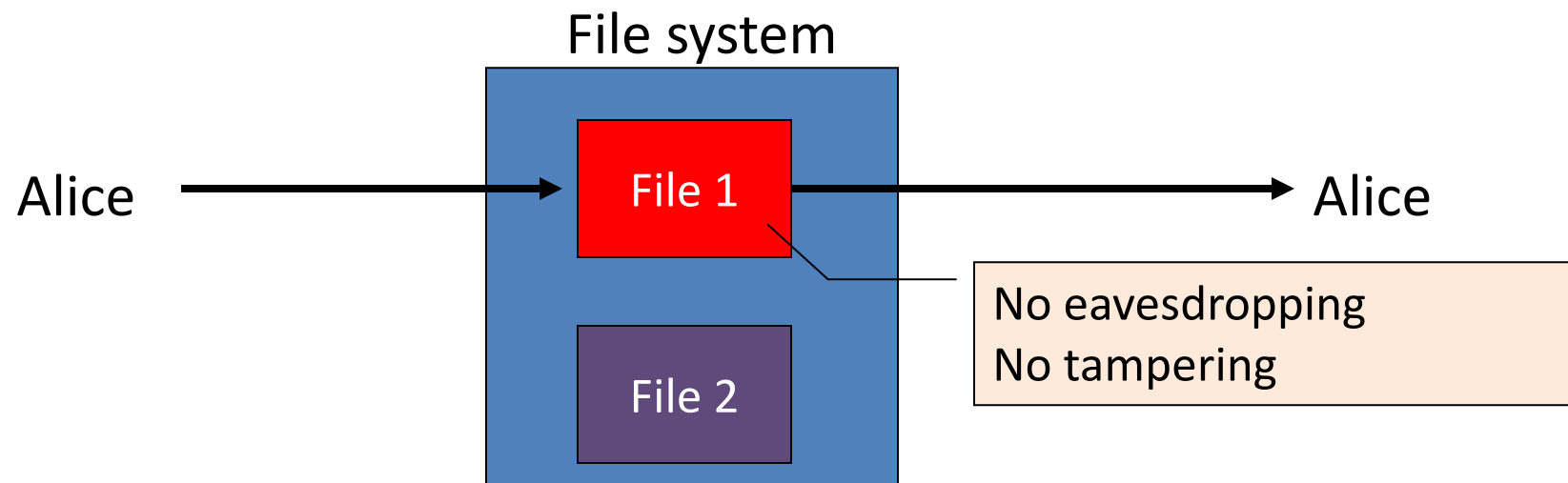
Diffie-Hellman key exchange

- Example

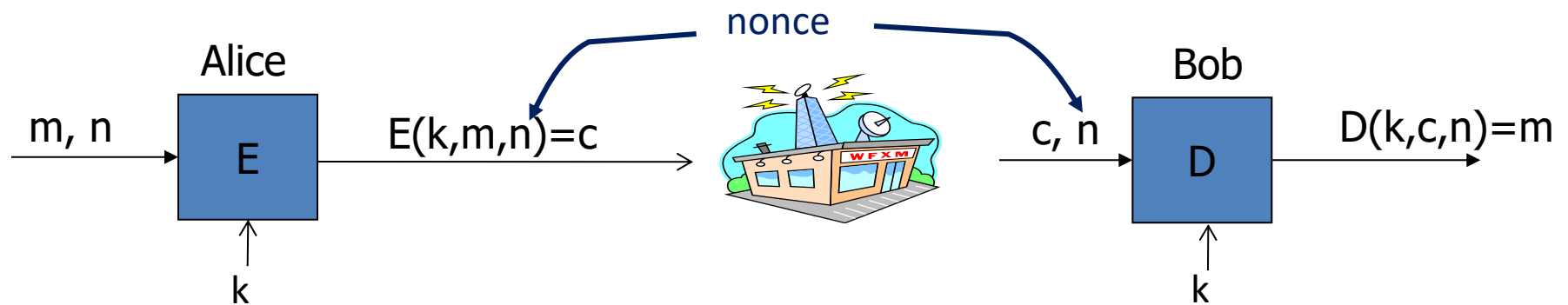


Goal 2: protected files

(protecting data at rest)



Building block: symmetric cipher



E, D: cipher **k: secret key (e.g. 128 bits)**

m, c: plaintext, ciphertext n: nonce (non-repeating)

Encryption algorithm is **publicly known**

⇒ never use a proprietary cipher

Use Cases

Single use key: (one time key)

- Key is only used to encrypt one message
 - encrypted email: new key generated for every email
- No need for nonce (set to 0)

Multi use key: (many time key)

- Key is used to encrypt multiple messages or multiple files
 - TLS: same key used to encrypt many frames
- Use either a *unique* nonce or a *random* nonce

First example: One Time Pad (single use key)

Vernam (1917)

Key:	0	1	0	1	1	1	0	0	1	0
Plaintext:	1	1	0	0	0	1	1	0	0	0
Ciphertext:	1	0	0	1	1	0	1	0	1	0

Encryption: $c = E(k, m) = m \oplus k$

Decryption: $D(k, c) = c \oplus k = (m \oplus k) \oplus k = m$

One Time Pad (OTP) Security

Shannon (1949):

- OTP is “secure” against one-time eavesdropping
- without key, ciphertext reveals no “information” about plaintext

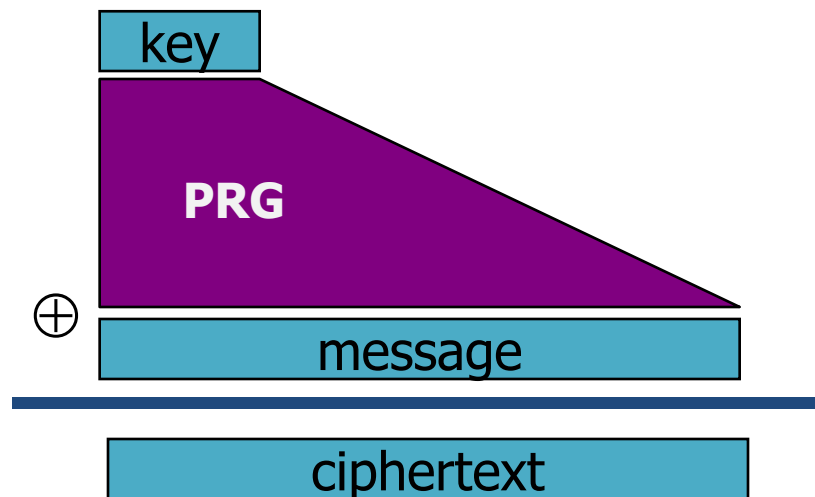
Problem: OTP key is as long as the message

Stream ciphers

(single use key)

Problem: OTP key is as long as the message

Solution: Pseudo random key -- stream ciphers



$$c \leftarrow \mathbf{PRG}(k) \oplus m$$

Example: **ChaCha20** (one-time if no nonce) key: 128 or 256 bits.

Dangers in using stream ciphers

One time key !! “Two time pad” is insecure:

$$c_1 \leftarrow m_1 \oplus \text{PRG}(k)$$

$$c_2 \leftarrow m_2 \oplus \text{PRG}(k)$$

What if want to use
same key to encrypt
two files?

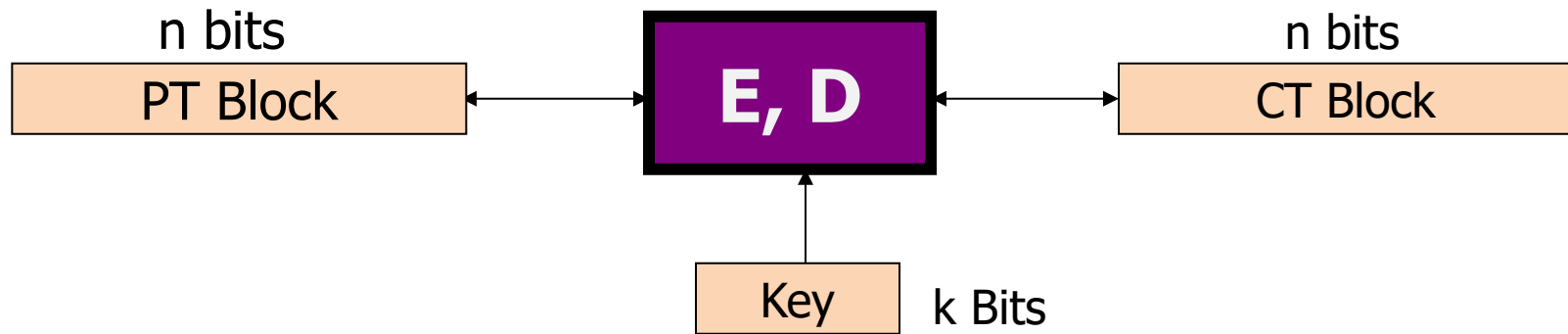
Eavesdropper does:

$$c_1 \oplus c_2 \rightarrow m_1 \oplus m_2$$

Enough redundant information in English that:

$$m_1 \oplus m_2 \rightarrow m_1, m_2$$

Block ciphers: crypto work horse

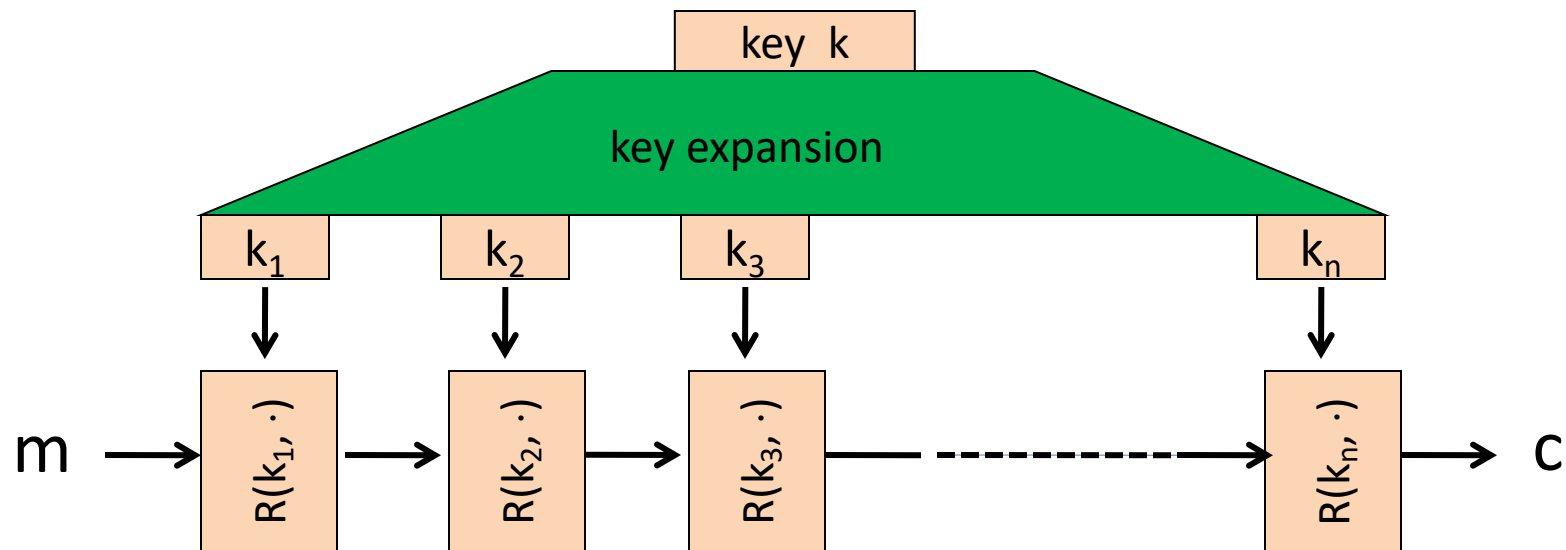


Canonical examples:

1. 3DES (old): $n = 64$ bits, $k = 168$ bits

2. AES: $n = 128$ bits, $k = 128, 192, 256$ bits

Block Ciphers Built by Iteration



$R(k, m)$: round function

for AES-128: 10 rounds, AES-256: $n=14$ rounds

AES-NI: AES in hardware (Intel, AMD, ARM)

New x86 hardware instructions used to implement AES:

- **aesenc, aesenclast**: one round of AES

aesenc xmm1, xmm2 (result written to xmm1)
state round key

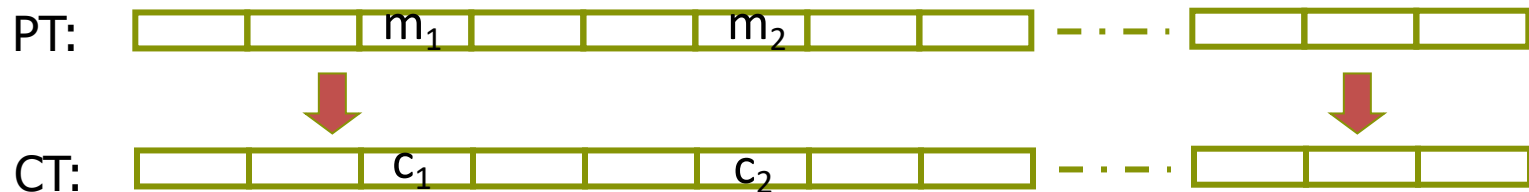
- **aesdec, aesdeclast**: one round of AES
- **aeskeygenassist**: do AES key expansion

⇒ more than 10x speedup over a software AES

⇒ better security: all AES instructions are constant time

Incorrect use of block ciphers

Electronic Code Book (ECB):



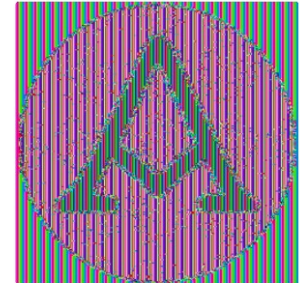
Problem:

– if $m_1 = m_2$ then $c_1 = c_2$

- Encryption
 - With ECB



test.bmp

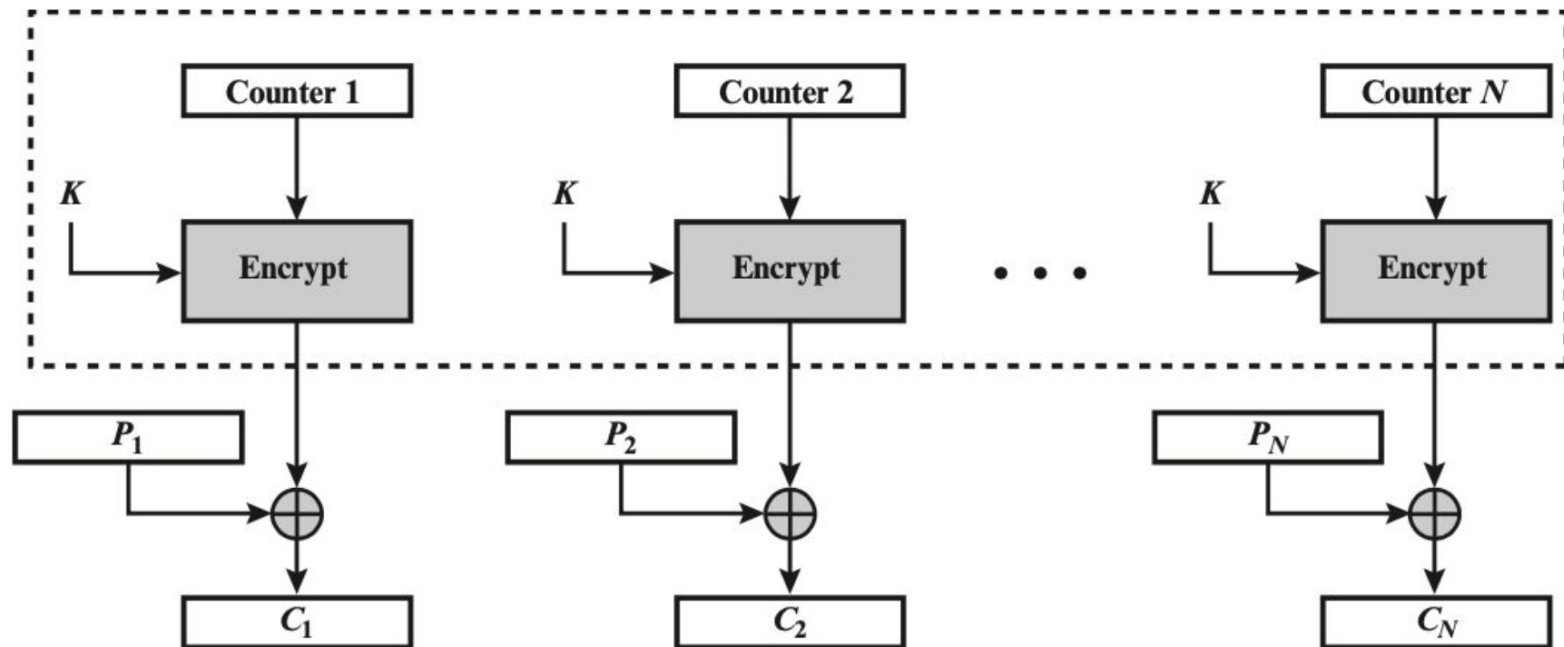


ecb.bmp

```
$ head -c 54 test.bmp > header.bin      # First 54 bytes of .bmp => header information

$ openssl enc -e -des-ecb -in test.bmp -out ecb1.bmp -K 0123456789012345 -iv 0000000000000000 -nopad
$ ls -l test.bmp ecb1.bmp
-rw-r--r--  1 ankit  staff  196672 Feb  5 22:11 ecb1.bmp
-rw-r--r--  1 ankit  staff  196672 Feb  5 20:56 test.bmp
$ tail -c 196618 ecb1.bmp > ecb2.bmp
$ cat header.bin ecb2.bmp > ecb.bmp
$ rm ecb1.bmp ecb2.bmp
$ ls -l test.bmp ecb.bmp
-rw-r--r--  1 ankit  staff  196672 Feb  5 22:11 ecb.bmp
-rw-r--r--  1 ankit  staff  196672 Feb  5 20:56 test.bmp
```

CTR mode encryption (eavesdropping security)



Why is this secure for multiple messages?

A Warning

eavesdropping security is insufficient for most applications

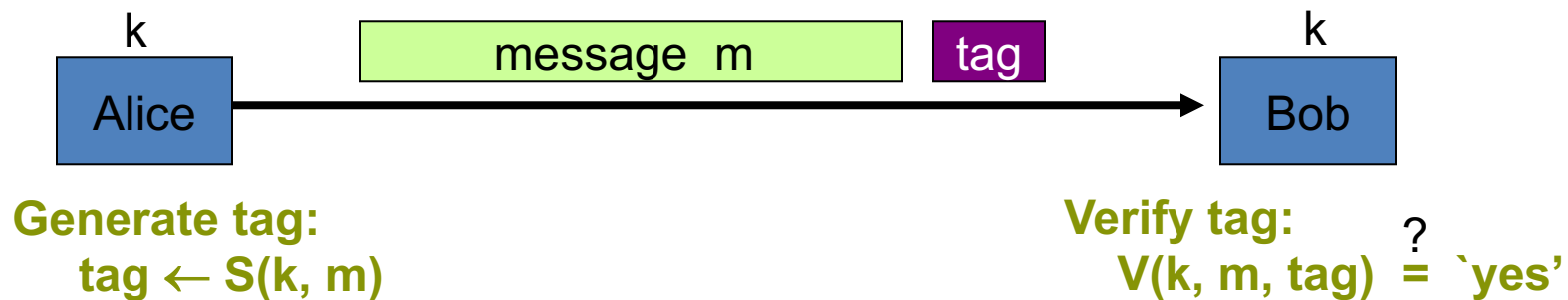
Need also to defend against active (tampering) attacks.

CTR mode is insecure against active attacks!

Next: methods to ensure message integrity

Message Integrity: MACs

- Goal: provide message integrity. No confidentiality.
 - ex: Protecting public binaries on disk.



Construction: HMAC (Hash-MAC)

Most widely used MAC on the Internet.

H: hash function.

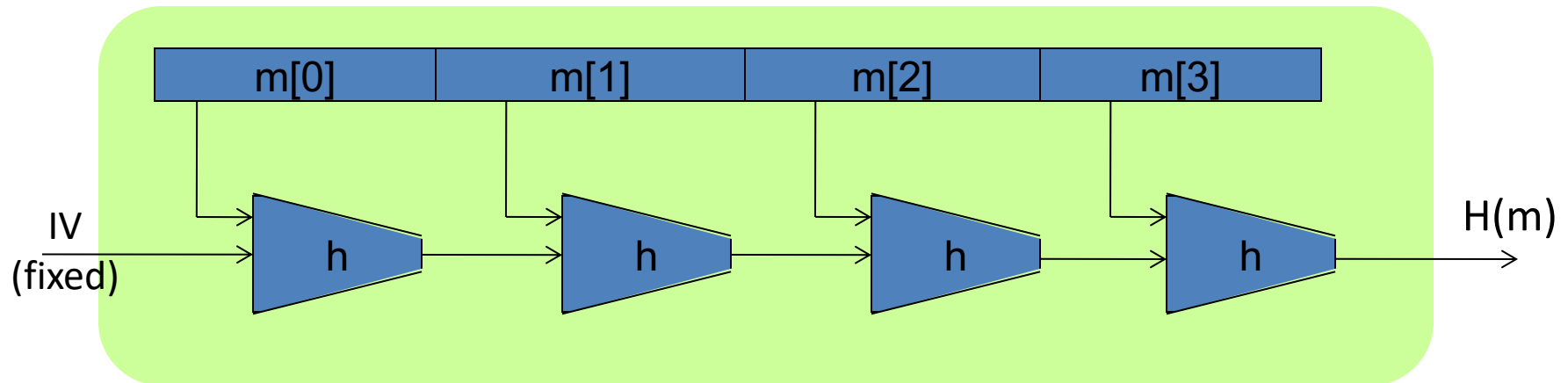
example: SHA-256 ; output is 256 bits

Building a MAC out of a hash function:

– Standardized method: HMAC

$$S(k, \text{msg}) = H(k \oplus \text{opad} \parallel H(k \oplus \text{ipad} \parallel \text{msg}))$$

SHA-256: Merkle-Damgard



- $h(t, m[i])$: compression function
- Thm 1: if h is collision resistant then so is H
- “Thm 2”: if h is a “PRF” then HMAC is a secure MAC

Why is this MAC construction secure?

... see PoIS

Combining MAC and ENC (Auth. Enc.)

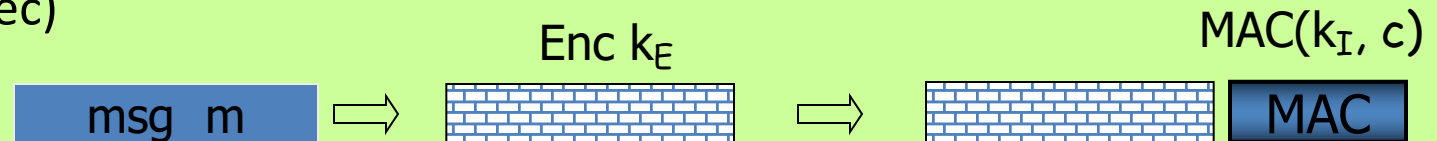
Encryption key k_E . MAC key = k_I

Option 1: (SSL)

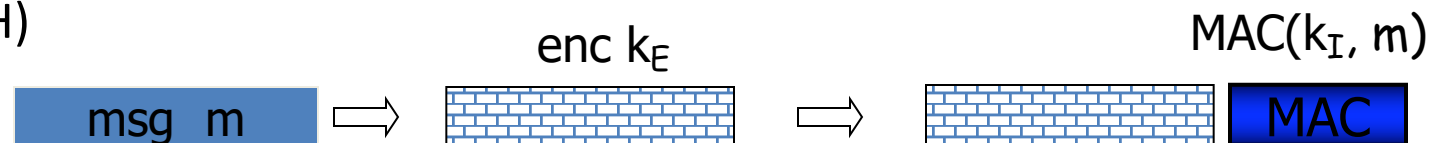


Option 2: (IPsec)

**always
correct**

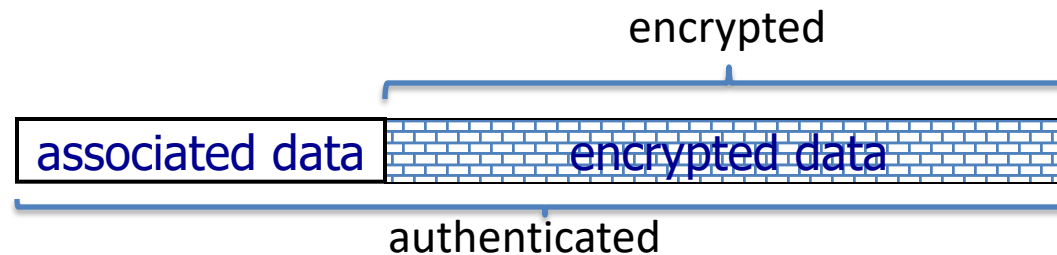


Option 3: (SSH)



AEAD: Auth. Enc. with Assoc. Data

AEAD:



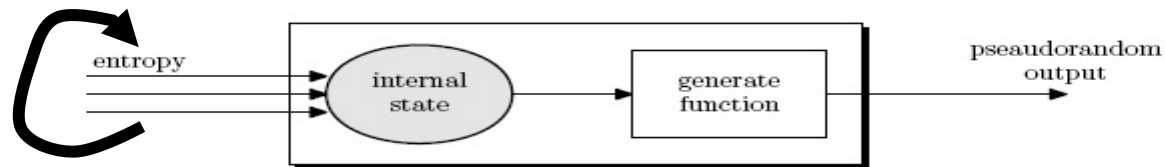
AES-GCM: CTR mode encryption then MAC

(MAC accelerated via Intel's PCLMULQDQ instruction)

Example AES-GCM encryption function

```
int encrypt(  
    unsigned char *key,                // key  
    unsigned char *iv,  int iv_len,    // nonce  
    unsigned char *plaintext,  int plaintext_len, // plaintext  
    unsigned char *aad,  int aad_len,    // assoc. data  
      
    unsigned char *ciphertext        // output ct  
)
```

Generating Randomness (e.g. keys, nonces)



Pseudo random generators in practice: (e.g. `/dev/random`)

- Continuously add entropy to internal state
- Entropy sources:
 - Hardware RNG: Intel **RdRand** inst. (Ivy Bridge). 3Gb/sec.
 - Timing: hardware interrupts (keyboard, mouse)

Summary

Shared secret key:

- Used for secure communication and document encryption

Encryption: (eavesdropping security) **[should not be used standalone]**

- One-time key: ex: a stream cipher
- Many-time key: ex: AES-CTR with a unique/random nonce

Integrity: HMAC

Authenticated encryption: encrypt-then-MAC using AES-GCM

Crypto Concepts

encryption and
compression problems

Encryption and compression: oil and vinegar

HTTP: uses compression to reduce bandwidth

Option 1: first encrypt and then compress

- Does not work ... ciphertext looks like a random string

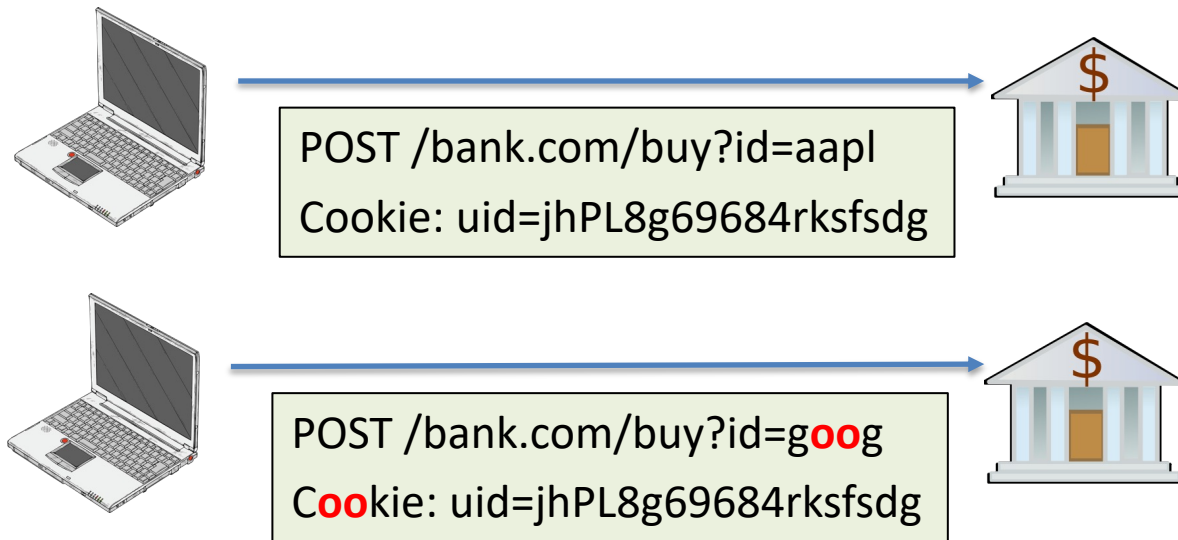
Option 2: first compress and then encrypt

- Used in many Internet protocols (TLS, HTTP, QUIC, ...)
- Trouble ...

Trouble ...

[Kelsey'02]

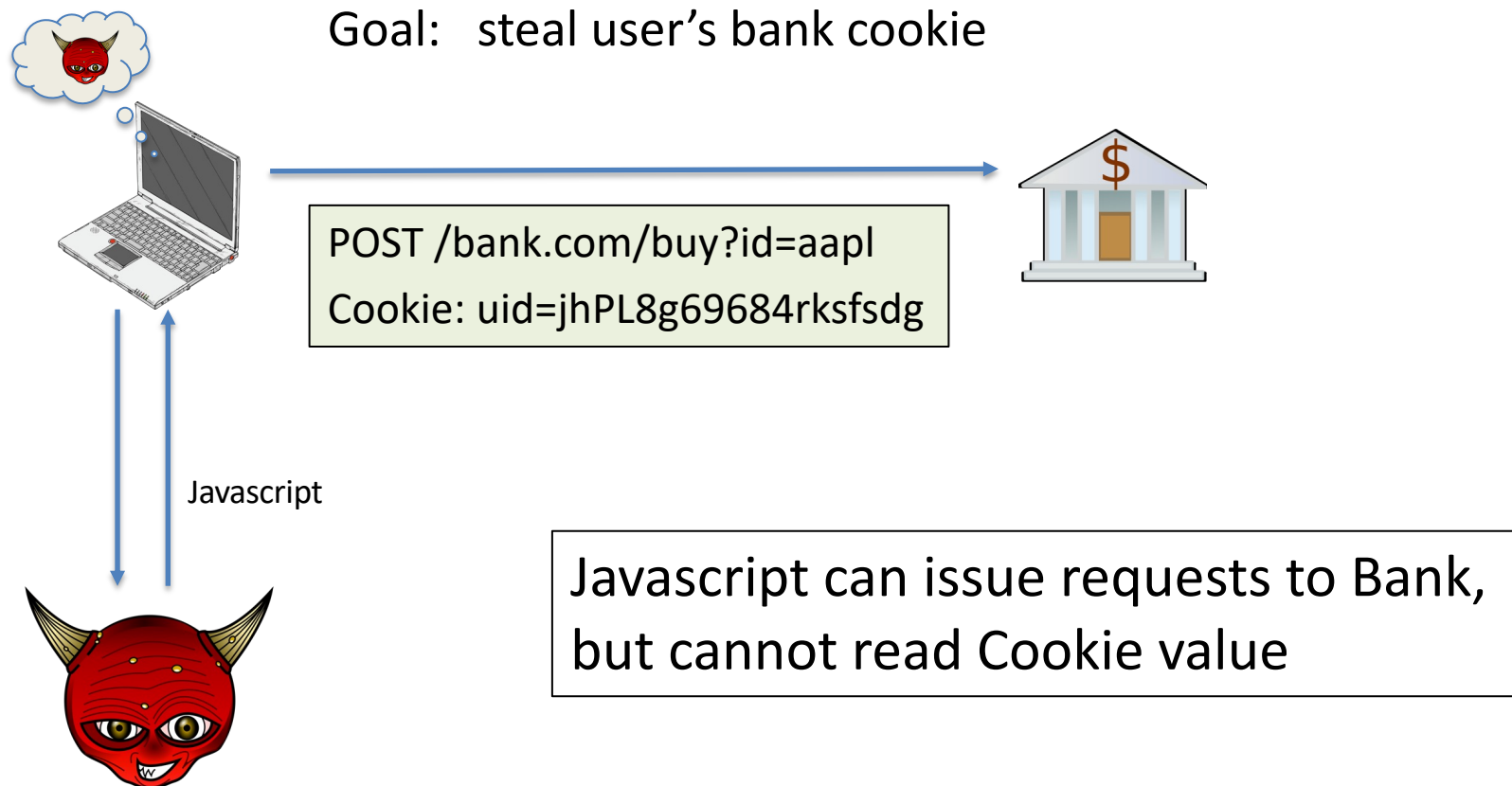
Compress-then-encrypt reveals information:



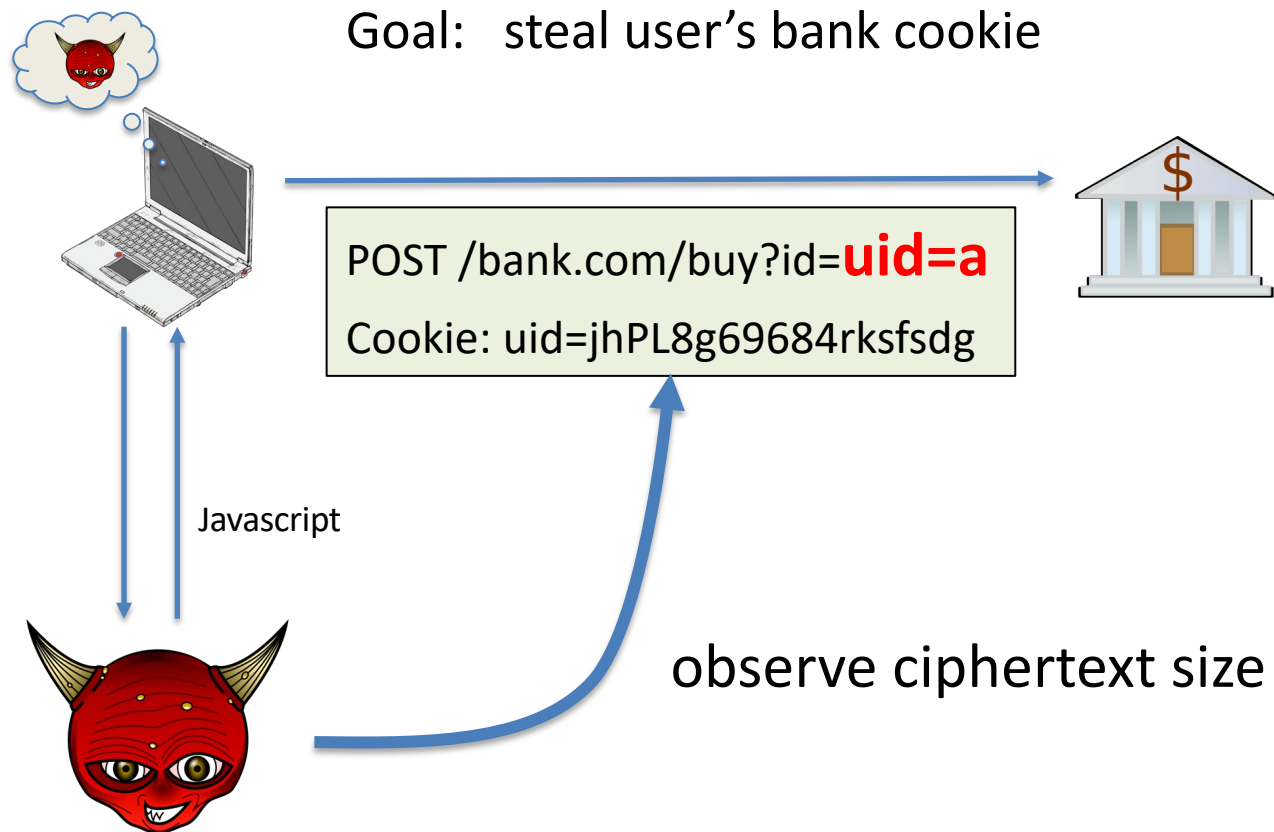
Second message compresses better than first:

network observer can distinguish the two messages!

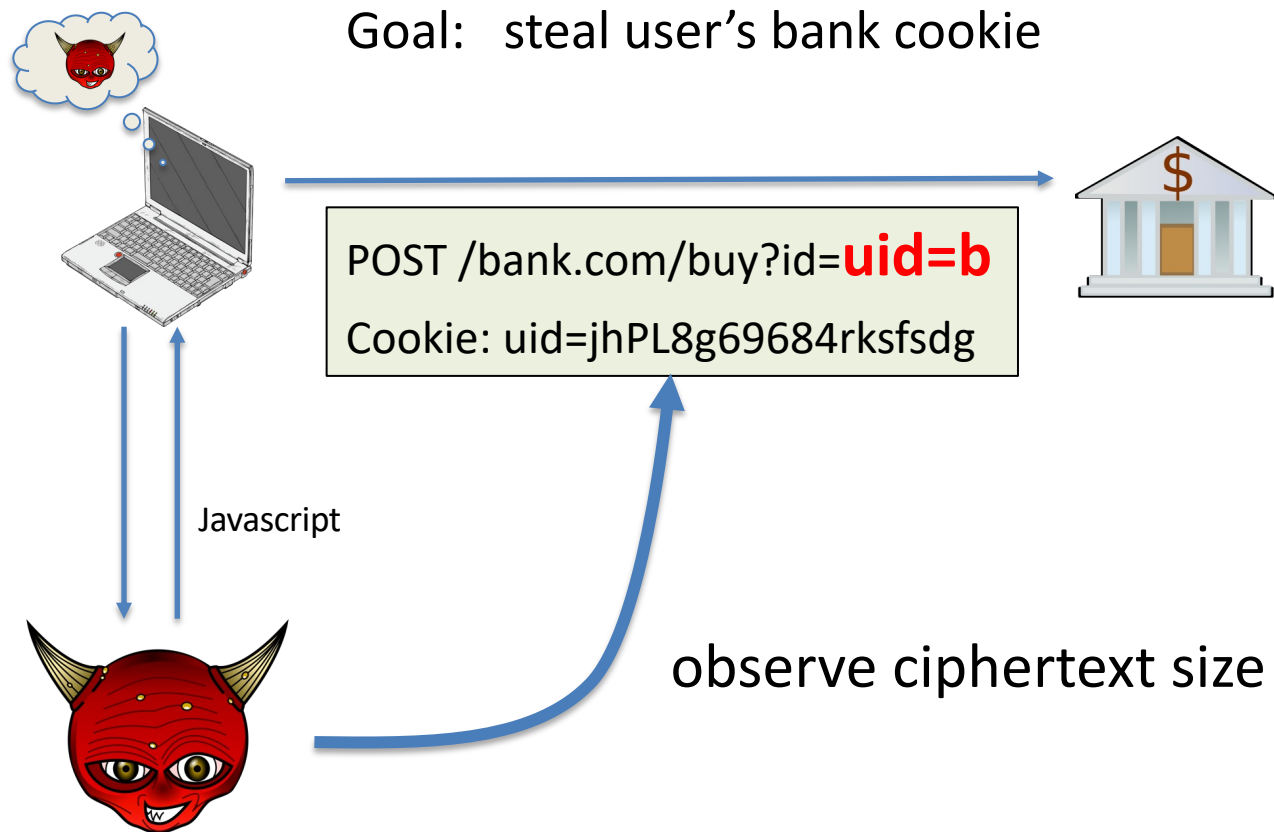
Even worse: the CRIME attack [RD'2012]



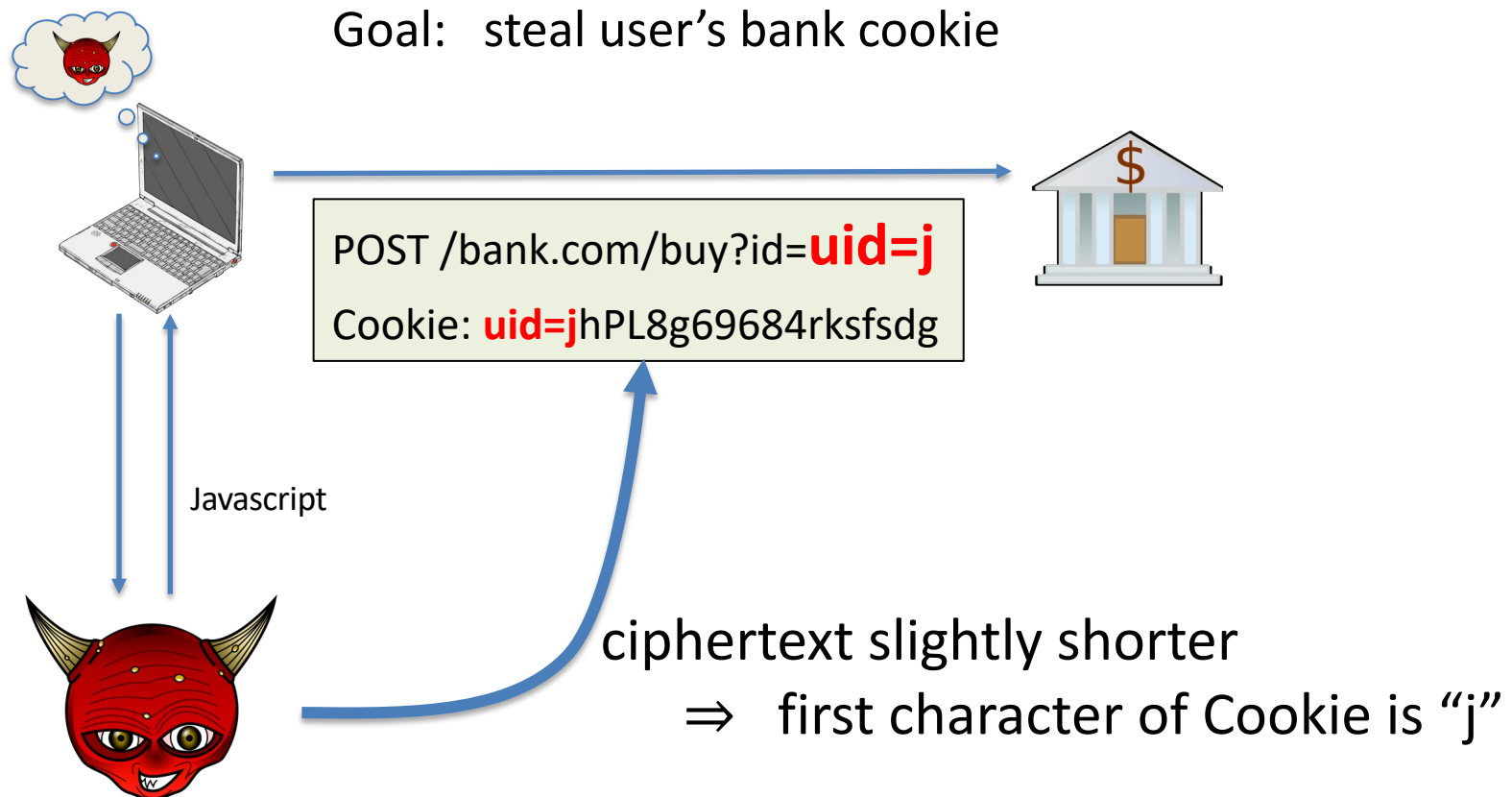
Even worse: the CRIME attack [RD'2012]



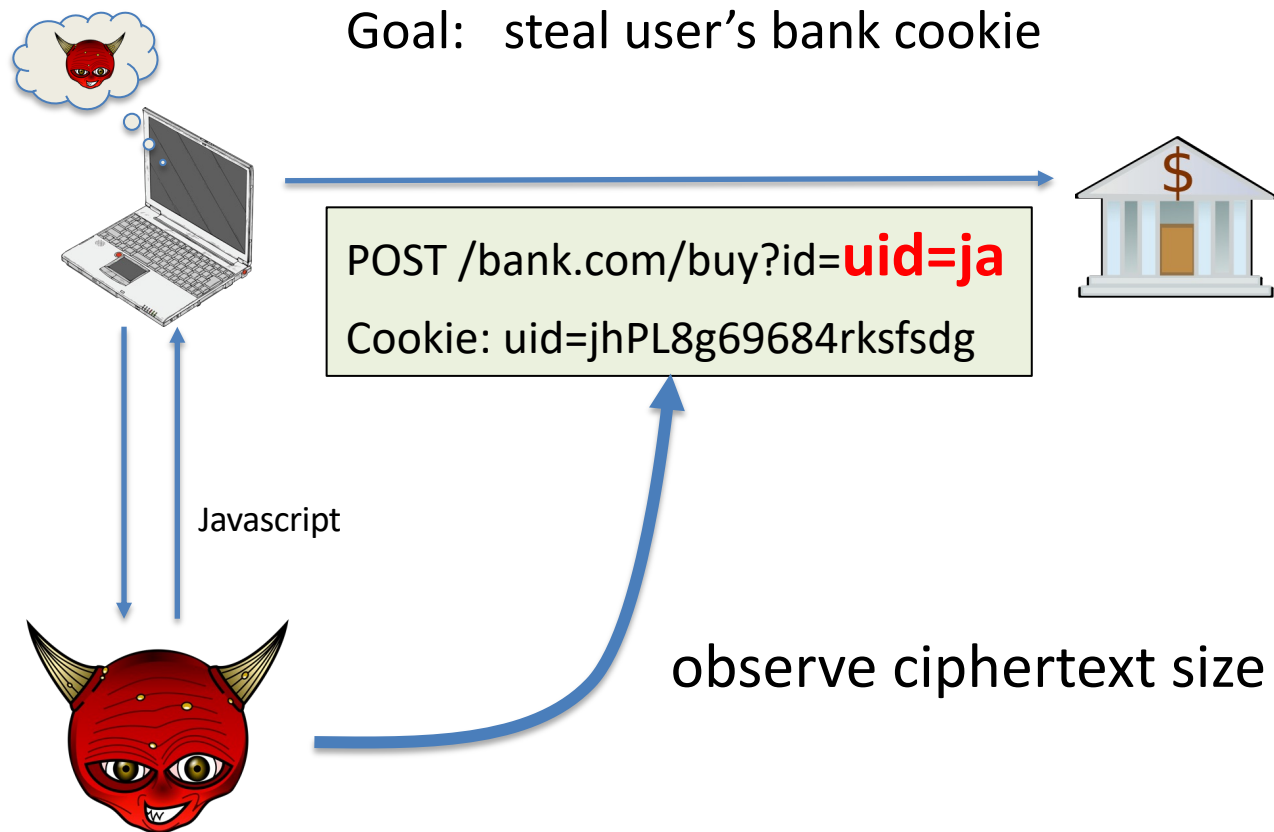
Even worse: the CRIME attack [RD'2012]



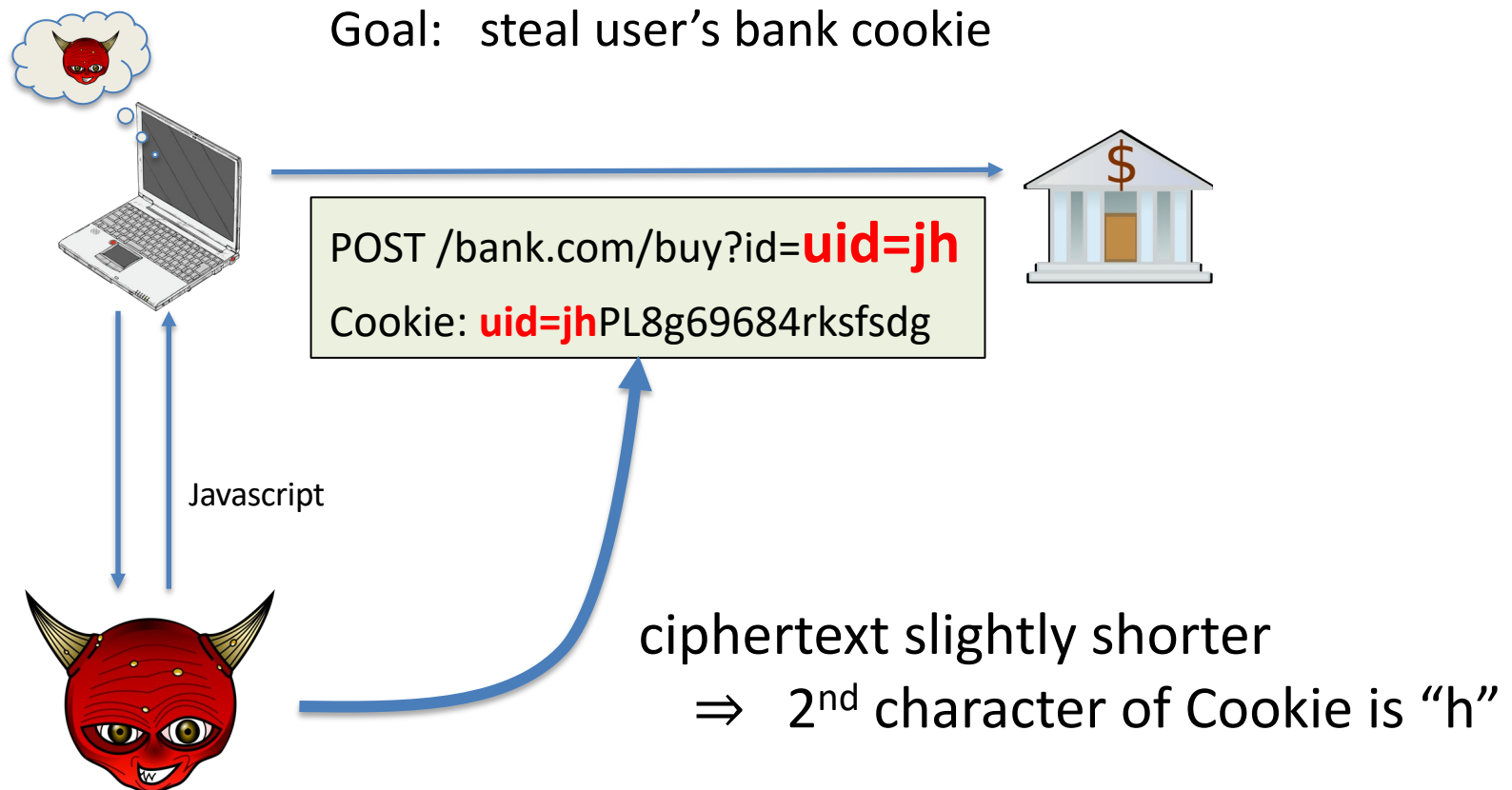
Even worse: the CRIME attack [RD'2012]



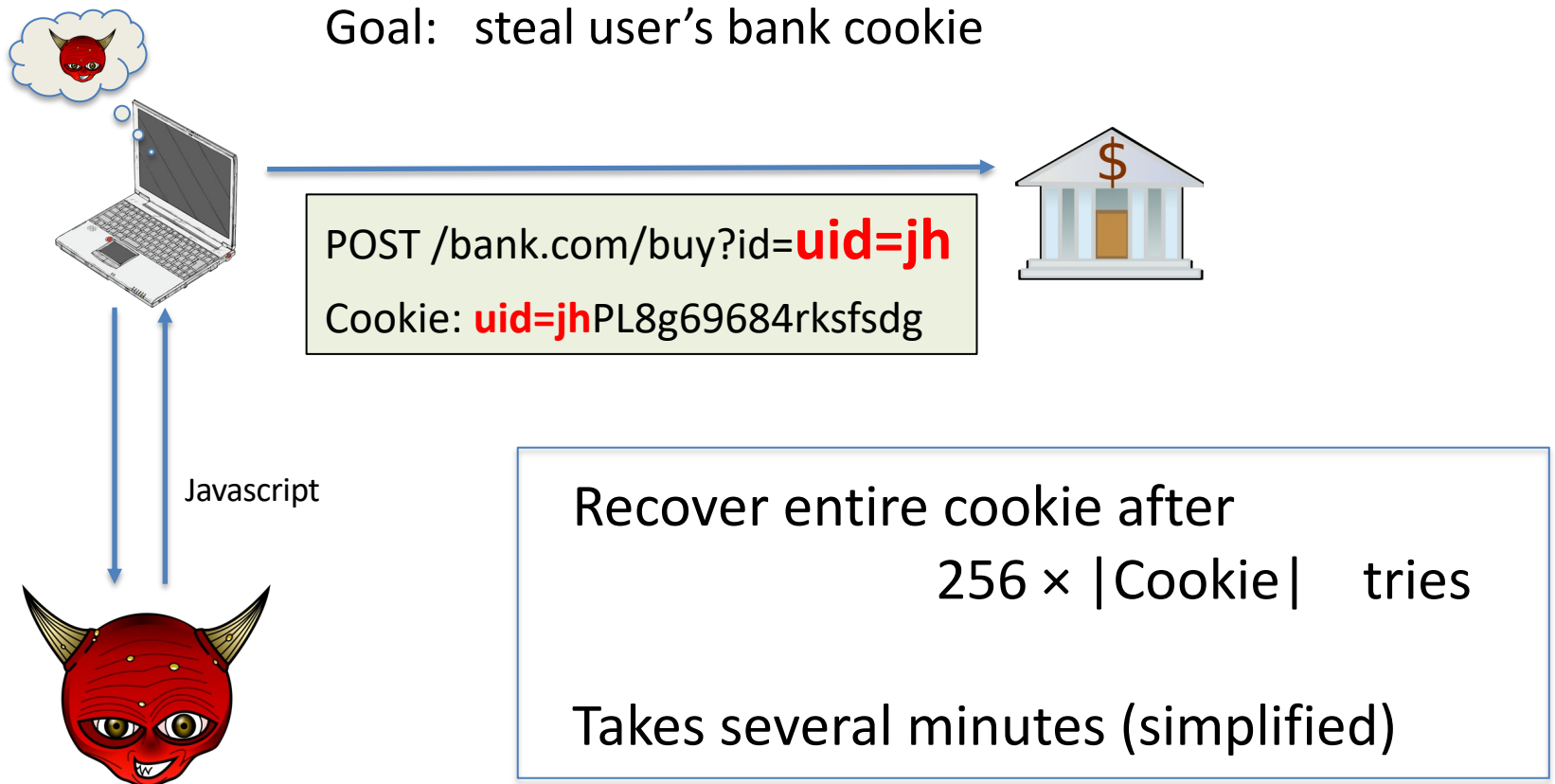
Even worse: the CRIME attack [RD'2012]



Even worse: the CRIME attack [RD'2012]

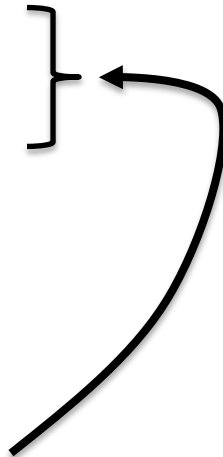


Even worse: the CRIME attack [RD'2012]



What to do?

- Disable compression 😞
- Use a different compression context for parts under Javascript control and parts that are not
- Change secret (Cookie) after every request



Does not eliminate inherent leakage due to compression